UNIT KOMPETENSI 11 • TI.VC.11.2026

MENGGUNAKAN SOURCE CODE VERSIONING

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Penggunaan Git/Repository • Alur kerja branching • Merging • Commit • Push/Pull • Cloning		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Menggunakan Source Code Versioning** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Penggunaan Git/Repository**: Menginisialisasi repositori Git lokal dan sinkronisasi dengan remote GitHub (AdityaRistantoMukti/Website_Intersis), konfigurasi .gitignore.
- **Alur kerja branching**: Menerapkan alur branching Git-Flow / Feature Branching (master/main produksi, fitur modular dan hotfix).
- **Merging**: Mengintegrasikan perubahan kode, fast-forward merge, pengelolaan merge conflict, dan validasi pasca merger.
- **Commit**: Menulis commit atomik dengan pesan berstandar Conventional Commits (feat, fix, docs, style, refactor).
- **Push/Pull**: Menyinkronkan delta commit ke remote GitHub dan memicu pipeline CI/CD Continuous Deployment Vercel & Railway.
- **Cloning**: Menduplikasi proyek ke lingkungan baru, konfigurasi dependensi, dan restorasi skema basis data dari fresh state.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Selamat pagi/siang Dewan Penguji yang terhormat. Dalam pengembangan perangkat lunak modern, manajemen versi berkas kode sumber adalah jaminan atas integritas, histori perubahan, dan kelancaran kolaborasi tim. Pada unit kompetensi **Menggunakan Source Code Versioning**, saya akan memperlihatkan alur kerja Git profesional yang diterapkan pada repositori proyek ini."

[Menjelaskan Sub 1: Penggunaan Git/Repository] "Pertama, *Penggunaan Git & Repositori*. Proyek ini diinisialisasi dan dilacak penuh menggunakan Git dengan remote repository publik di GitHub pada alamat `github.com/AdityaRistantoMukti/Website_Intersis`. Kami mengonfigurasi berkas `.gitignore` secara disiplin untuk memastikan direktori berat seperti `node_modules/`, folder artefak kompilasi `dist/`, serta berkas sensitif `.env` tidak pernah terunggah ke repositori publik demi keamanan kredensial."

[Menjelaskan Sub 2, 3, & 4: Branching, Merging, dan Commit] "Kedua, mengenai *Branching, Merging, dan Commit*. Kami memisahkan pengembangan fitur ke dalam branch kerja sebelum digabungkan ke branch utama `master`. Setiap kali penggabungan (*Merging*) dilakukan, kami memeriksa potensi konflik dan memastikan pengujian fungsional lolos. Kami menerapkan standar **Conventional Commits** yang ketat pada setiap commit history, seperti `feat(about): enhance layout spacing`, `fix(mobile): responsive fixes for hero controls`, dan `docs: add technical handbook`. Pesan commit yang atomik dan deskriptif mempermudah penelusuran audit perubahan kapan pun dibutuhkan."

[Menjelaskan Sub 5 & 6: Push/Pull dan Cloning] "Ketiga, *Push/Pull dan Cloning*. Setiap kali perintah `git push origin master` dieksekusi, webhook GitHub secara otomatis memicu pipeline **Continuous Deployment (CD)** ke platform Vercel untuk frontend dan Railway untuk backend. Terakhir, repositori ini dirancang 100% *reproducible*: pengembang lain dapat melakukan `git clone`, menjalankan `npm install`, menyalin file `.env.example`, mengeksekusi seeder database, dan aplikasi langsung siap berjalan dalam waktu kurang dari 5 menit."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

11.1 & 11.4. Konfigurasi Repositori Git & Standarisasi Conventional Commits

Kaitkan ke Codebase: `.gitignore & Git Commit Log`

Catatan log commit membuktikan kedisiplinan penggunaan standar Conventional Commits pada proyek ini:

```
Git Commit Log History (git log -n 5 --oneline)
e31edd3 fix(home): dynamic redirection for hero action buttons to products and contact
0ea54a5 docs: add technical handbook and glossary PDF for internship defense
d05a057 fix(mobile): responsive fixes for hero controls, padding, monochrome stats, theme icon
f4e15bd feat: track real product uploads, add database backup, and sync utilities
467fc25 setup Vercel & Railway
```

11.2 & 11.3. Strategi Alur Kerja Branching & Resolusi Konflik (Merging)

Kaitkan ke Codebase: `Git Branching Workflow`

Siklus hidup isolasi fitur dan penggabungan kode sumber:

```
Alur Branching & Merging
[master / production] ------------------------------------------------> [Deployment Vercel/Railway]
          ^                                          |
          |  (git merge --no-ff)                     | (git pull)
          |                                          v
[feature/products-module] -----> [Commit 1] -> [Commit 2] -+
```

```
Terminal Commands: Branching & Merging Cycle
# Membuat branch fitur baru yang terisolasi
git checkout -b feat/products-module

# Mengembangkan kode dan melakukan commit atomik
git add .
git commit -m "feat(products): create responsive bento product showcase"

# Berpindah ke master dan menggabungkan perubahan
git checkout master
git merge feat/products-module

# Menghapus branch fitur lokal pasca merge sukses
git branch -d feat/products-module
```

11.5 & 11.6. Sinkronisasi Cloud Push/Pull & Prosedur Replikasi Cloning

Kaitkan ke Codebase: `Panduan Cloning & Eksekusi Proyek`

Panduan langkah demi langkah mereplikasi seluruh proyek di mesin baru dari fresh clone:

Tahapan	Perintah Shell	Luaran yang Dihasilkan
1. Kloning Repositori	`git clone https://github.com/AdityaRistantoMukti/Website_Intersis.git`	Menduplikasi seluruh source code dan commit history lengkap ke lokal disk.
2. Instalasi Dependensi	`cd backend && npm install && cd ../frontend && npm install`	Mengunduh seluruh pustaka yang terkunci di package-lock.json.
3. Inisialisasi Database	`mysql -u root -p < database/isa_career_hrd.sql`	Membuat 12 tabel relasional lengkap dengan data awal (seeders).
4. Menjalankan Layanan	`npm run dev` (di backend dan frontend)	Website interaktif siap diakses di http://localhost:5173.

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Pelacakan Repositori Penuh: Terkelola di GitHub 'AdityaRistantoMukti/Website_Intersis'.
- Proteksi .gitignore: Mengabaikan node_modules, build bundle, dan berkas kredensial .env.
- Standar Conventional Commits: Penggunaan prefix feat, fix, docs, style yang konsisten.
- Alur Branching & Merging: Isolasi fitur independen sebelum digabungkan ke cabang produksi.
- Otomasi CI/CD & Reproducibility: Auto-deploy ke Vercel/Railway dan kemudahan fresh cloning.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Bagaimana strategi Anda dalam menyelesaikan Merge Conflict saat ada perubahan bertabrakan?

Jawaban Pembelaan Presenter: Ketika terjadi merge conflict, Git menandai baris yang bertabrakan dengan marker <<<<<<< HEAD, =======, dan >>>>>>>. Kami melakukan inspeksi manual bersama rekan tim untuk menganalisis baris logika mana yang harus dipertahankan, membersihkan marker konflik, menjalankan pengujian unit/sistem lokal untuk memastikan aplikasi tetap berjalan, lalu melakukan commit resolusi merge.

Q2: Mengapa penting menulis pesan commit dengan pola Conventional Commits?

Jawaban Pembelaan Presenter: Conventional Commits memformat pesan commit menjadi terstruktur mesin dan manusia (misal feat untuk fitur baru, fix untuk perbaikan bug). Hal ini memudahkan pelacakan changelog otomatis, pemahaman riwayat perubahan oleh tim, dan memfasilitasi penentuan kenaikan nomor versi Semantic Versioning secara otomatis.